WEEK 6:

Lecture 26: Ontological Engineering & Knowledge Representation

This document discusses ontological engineering and knowledge representation, focusing on general and flexible representations for complex real-world domains. It covers several key concepts:

● **Ontological Engineering**

The process of representing general concepts like events, time, and physical objects, forming an "upper ontology" with general concepts at the top and more specific ones below.

● **Characteristics of General-Purpose Ontology**

Such an ontology should be applicable across various specialized domains, unifying different areas of knowledge for reasoning and problem-solving.

● **Categories and Objects**

Emphasizes organizing objects into categories for knowledge representation, which is crucial for reasoning and making predictions. It explains how categories can be represented using predicates or as reified objects in first-order logic, and how inheritance simplifies knowledge bases through taxonomic hierarchies.

● **Types of Facts about Categories**

Illustrates how to state facts about categories, including object membership, subcategory relations, properties of category members, and properties of a category as a whole.

● **Disjoint Categories, Exhaustive Decompositions, and Partitions**

Defines these concepts for managing relationships between categories, especially when they are not subclasses of each other.

● **Physical Composition**

Discusses the "PartOf" relation for representing that one object is a component of another, noting its transitivity and reflexivity. It also introduces the concept of "bunches" for composite objects with definite parts but no particular structure, especially when dealing with properties like weight.

● **Measurements**

Explains how to represent quantitative measures (like length, diameter, cost) using units and axioms for conversion. It also addresses qualitative measures (like difficulty or beauty), emphasizing their orderability even without numerical values.

● **Objects (Stuff vs. Things)**

Distinguishes between "things" (discrete objects like a zebra) and "stuff" (portions of reality that defy obvious individuation, like butter). It highlights that intrinsic properties (density, color) are retained upon subdivision of stuff, while extrinsic properties (weight, shape) are not.

● **Events**

Introduces event calculus as an alternative to situation calculus to handle continuous and simultaneous actions. It explains how fluents and events are reified, and how predicates like Happens, Initiates, and Terminates are used to describe events and their effects on fluents over time intervals.

Lecture 27: Truth Maintenance Systems (TMS)

This document discusses Truth Maintenance Systems (TMS) and related concepts in artificial intelligence, focusing on how knowledge representation systems handle non-monotonic reasoning and belief revision.

● **Processes and Time Intervals**

It introduces the distinction between discrete events (with a definite beginning, middle, and end) and process categories (which hold true for any subinterval). It also defines different types of time intervals (moments and extended intervals) and explains interval relations like "Meet," "Before," and "Overlap."

● **Mental Events**

The document explores the need for agents to have knowledge about their own and other agents' beliefs and reasoning processes. It highlights the problem of "referential transparency" in standard logic when dealing with propositional attitudes like "Believes" and "Knows," and the desire for "referential opacity" in such cases.

● **Non-monotonic Reasoning**

This section explains that human common-sense reasoning often involves "jumping to conclusions" by default, which can be retracted with new evidence. This behavior is called non-monotonicity, where beliefs don't grow monotonically over time.

● **Circumscription**

One approach to non-monotonic reasoning, circumscription, allows specifying predicates to be "as false as possible." It enables drawing default conclusions (e.g., "birds fly") unless specific exceptions are known. It's viewed as a model preference logic where a sentence is entailed if it's true in all preferred models (those with fewer abnormal objects). The "Nixon diamond" example illustrates how it handles conflicting defaults.

● **Default Logic**

Another formalism for non-monotonic reasoning, default logic uses default rules of the form "P: J1,...,Jn/C," meaning if P is true and J1 to Jn are consistent with the knowledge base, then C can be concluded by default. The "Nixon diamond" is also used to demonstrate default logic, leading to multiple possible "extensions" (maximal sets of consequences).

● **Truth Maintenance System (TMS)**

This section details the core concept of TMS. It addresses the problem of belief revision, where inferred facts might be wrong and need to be retracted without introducing inconsistencies. TMSs are designed to manage these complications efficiently, especially when a retracted sentence P has led to other inferences (Q), and those inferences might have other independent justifications.

● **Simple TMS Approach**

A basic method involves tracking the order of sentences. Retracting a sentence means reverting the system to the state before that sentence was added, then re-adding subsequent sentences. This is simple but impractical for large knowledge bases due to the computational cost.

● **Justification-Based Truth Maintenance System (JTMS)**

A more efficient approach where each sentence is annotated with a justification (the set of sentences from which it was inferred). Retracting a sentence P in JTMS only deletes sentences for which P is a member of all their justifications. Sentences are marked "in" or "out" rather than entirely deleted to retain inference chains.

● **Assumption-Based Truth Maintenance System (ATMS)**

This system enhances efficiency for context switching between hypothetical worlds. Unlike JTMS which represents one state at a time, ATMS represents all considered states simultaneously. For each sentence, it tracks which sets of assumptions would cause it to be true (a label consisting of assumption sets). ATMS also helps in generating explanations for a sentence, which can include assumptions.

● **TMS Node States and Justifications**

In a general TMS, each statement or rule is a "node" and is either "IN" (believed true) or "OUT" (not believed true). IN nodes have at least one valid justification, while OUT nodes have none.

○ **Support List (SL)**

A statement is valid if all "in-nodes" are IN and all "out-nodes" are OUT.

○ **Conditional Proof (CP)**

Represents hypothetical arguments, where the consequent is IN if "in-hypothesis" nodes are IN and "out-hypothesis" nodes are OUT.

● **Examples**

The document provides examples of TMS usage, including simple logical deductions and a scheduling scenario involving contradictions.

Lecture 28: Representation Schemes – 1

This file discusses various knowledge representation schemes, focusing on Semantic Networks and introducing Description Logics.

● **Knowledge Representation Issues**

Covers topics like generality, specificity, definitions, exceptions, causality, uncertainty, and the choice of schemes (Semantic networks, scripts, frames, stochastic methods, connectionist) and implementation media (Prolog, Lisp, C, Java).

● **Semantic Networks**

- Represent knowledge as a graph with labeled nodes (objects/categories) and arcs (relations/associations).
- Facilitate inheritance reasoning.
- Handle default values and exceptions.
- Efficient compared to logical theorem proving.
- Limitations: difficulty in representing full first-order logic.
- Procedural attachment can enhance expressive power.

● **Description Logics**

- Formalize semantic networks.
- Focus on taxonomic structure.
- Tasks: subsumption, classification, consistency checking.
- Emphasize tractability (polynomial time).
- CLASSIC language example provided.

Lecture 29: Representation Schemes – 2

● **Description Logics**

Formal notations for describing categories and their properties with emphasis on tractability.

● **Case Frames**

Verb-oriented representation defining roles (agent, object, instrument).

● **Conceptual Dependency**

Schank's theory using primitive conceptualizations (ACTs, PPs, AAs, PAs).

● **Scripts**

Structured representations of stereotyped event sequences (e.g., RESTAURANT script).

● **Frames**

Minsky's structured data representation with slots, defaults, and procedural info.

● **Conceptual Graphs**

Graph-based representation with concept nodes and relation nodes, supporting logical operations and inheritance.

Lecture 30: Acting Under Uncertainty

● **Difficulty in Handling Uncertainty**

Arises from partial observability and non-determinism.

● **Utility Theory**

States have utility; agents prefer higher utility outcomes.

● **Decision Theory**

Combines probability + utility → Maximum Expected Utility (MEU).

● **Probabilities**

- Prior (unconditional)
- Posterior (conditional)
- Formula:
  $P(a|b) = P(a \wedge b) / P(b)$
- Product rule:
  $P(a \wedge b) = P(a|b)P(b)$

● **Inference Using Full Joint Probability**

- Uses full joint distribution
- Marginalization and conditioning
- Normalization ensures probabilities sum to 1
- Limitation: exponential complexity

● **Independence**

- $P(a|b) = P(a)$
- Reduces complexity

● **Bayes' Rule**

- Formula:
  $P(b|a) = [P(a|b)P(b)] / P(a)$
- Used in diagnosis and probabilistic reasoning